管理應用程式執行掛勾

Astra Control Center

NetApp
November 21, 2023

This PDF was generated from https://docs.netapp.com/zh-tw/astra-control-center-2112/use/execution-hook-examples.html on November 21, 2023. Always check docs.netapp.com for the latest.

目錄

管理應用程式執行掛勾 1
預設執行掛勾和規則運算式 1
關於自訂執行掛勾的重要注意事項 1
檢視現有的執行掛勾 2
建立自訂執行掛勾 2
停用執行掛勾 3
刪除執行掛勾 3
執行攔截範例 3

管理應用程式執行掛勾

執行攔截是自訂指令碼、您可以在託管應用程式的快照之前或之後執行。例如、如果您有資料庫應用程式、您可以使用執行掛勾來暫停快照之前的所有資料庫交易、並在快照完成後繼續交易。如此可確保應用程式一致的快照。

預設執行掛勾和規則運算式

對於某些應用程式、Astra Control隨附NetApp提供的預設執行掛勾、可處理快照前後的凍結和解凍作業。Astra Control使用規則運算式、將應用程式的容器映像與下列應用程式配對：

- MariaDB
  - 符合規則運算式：\bmariadb\b
- MySQL
  - 相符的規則運算式：\bmysql\b
- PostgreSQL
  - 相符的規則運算式：\bpostgresql\b

如果有相符項目、則NetApp提供的該應用程式預設執行掛勾會顯示在應用程式的作用中執行掛勾清單中、而這些掛勾會在擷取該應用程式的快照時自動執行。如果其中一個自訂應用程式有類似的映像名稱、正好符合其中一個規則運算式（而且您不想使用預設的執行掛勾）、您可以變更映像名稱、 或停用該應用程式的預設執行掛勾、改用自訂掛勾。

您無法刪除或修改預設的執行掛勾。

關於自訂執行掛勾的重要注意事項

規劃應用程式的執行掛勾時、請考量下列事項。

- Astra Control需要以執行Shell指令碼的格式寫入執行掛勾。
- 指令碼大小限制為128 KB。
- Astra Control會使用執行掛勾設定和任何符合條件來判斷哪些掛勾適用於快照。
- 所有執行掛機故障都是軟性故障、即使掛機故障、仍會嘗試其他掛機和快照。但是、當掛機失敗時、會在*活動*頁面事件記錄中記錄警告事件。
- 若要建立、編輯或刪除執行掛勾、您必須是擁有擁有者、管理員或成員權限的使用者。
- 如果執行掛機執行時間超過25分鐘、掛機將會失敗、並建立傳回代碼為「N/A」的事件記錄項目。任何受影響的快照都會逾時並標示為故障、並會出現一個事件記錄項目、指出逾時時間。

> 由於執行掛勾通常會減少或完全停用執行中應用程式的功能、因此您應該一律盡量縮短自訂執行掛勾執行所需的時間。

執行快照時、執行掛機事件會依照下列順序進行：

1. 任何適用的NetApp提供的預設快照前執行掛勾、都會在適當的容器上執行。

2. 任何適用的自訂快照前執行掛勾都會在適當的容器上執行。您可以視需要建立及執行任意數量的自訂快照前置掛勾、但在快照之前執行這些掛勾的順序並不保證也無法設定。
3. 快照即會執行。
4. 任何適用的自訂快照後執行掛勾都會在適當的容器上執行。您可以視需要建立及執行任意數量的自訂快照後掛勾、但快照後執行這些掛勾的順序並不保證也無法設定。
5. 任何適用的NetApp提供的預設快照後執行掛勾、都會在適當的容器上執行。

> 在正式作業環境中啟用執行攔截指令碼之前、請務必先進行測試。您可以使用'kubecl exec'命令來方便地測試指令碼。在正式作業環境中啟用執行掛勾之後、請測試所產生的快照、以確保它們一致。您可以將應用程式複製到暫用命名空間、還原快照、然後測試應用程式、藉此完成此作業。

檢視現有的執行掛勾

您可以檢視現有的自訂或NetApp提供的應用程式預設執行勾點。

步驟

1. 移至*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。

   您可以在結果清單中檢視所有已啟用或已停用的執行掛勾。您可以查看Hook的狀態、來源、以及何時執行（快照前或快照後）。若要檢視執行掛起的相關事件記錄、請前往左側導覽區域的*活動*頁面。

建立自訂執行掛勾

您可以為應用程式建立自訂執行掛勾。請參閱 "執行攔截範例" 如需攔截範例、您需要擁有擁有擁有者、管理員或成員權限、才能建立執行掛勾。

> 當您建立自訂Shell指令碼作為執行掛勾時、請記得在檔案開頭指定適當的Shell、除非您執行Linux命令或提供執行檔的完整路徑。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*新增連結*。
4. 在* HookDetails（掛機詳細資料）區域中、視掛機的執行時間而定、選擇*預先快照*或*快照後。
5. 輸入掛機的唯一名稱。
6. （選用）輸入執行期間要傳遞至掛機的任何引數、並在您輸入的每個引數之後按Enter鍵以記錄每個引數。
7. 在「* Container images"（* Container映像*）區域中、如果掛勾應針對應用程式中包含的所有容器映像執行、請啟用「* Apply to all Container images"（套用至所有容器映像）核取方塊。如果掛機只能對一個或多個指定的容器映像起作用、請在「要比對的容器映像名稱」欄位中輸入容器映像名稱。
8. 在*指令碼*區域中、執行下列其中一項：

- 上傳自訂指令碼。
  i. 選取*上傳檔案*選項。
  ii. 瀏覽至檔案並上傳。
  iii. 為指令碼指定唯一名稱。
  iv. （選用）輸入其他系統管理員應該知道的任何指令碼附註。
- 從剪貼簿貼入自訂指令碼。
  i. 選擇*從剪貼簿貼上*選項。
  ii. 選取文字欄位、然後將指令碼文字貼到欄位中。
  iii. 為指令碼指定唯一名稱。
  iv. （選用）輸入其他系統管理員應該知道的任何指令碼附註。

9. 選取*新增攔截*。

停用執行掛勾

如果您想要暫時避免在應用程式快照之前或之後執行、可以停用執行掛勾。您需要擁有擁有擁有者、管理員或成員權限、才能停用執行掛勾。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*「動作*」下拉式清單以顯示您要停用的掛勾。
4. 選擇*停用*。

刪除執行掛勾

如果不再需要執行掛勾、您可以完全移除該掛勾。您需要擁有擁有擁有者、管理員或成員權限、才能刪除執行掛勾。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*「動作」*下拉式選單以選取您要刪除的掛勾。
4. 選擇*刪除*。

執行攔截範例

請使用下列範例、瞭解如何建立執行掛勾的架構。您可以使用這些hooks做為範本、或做為測試指令碼。

簡單的成功範例

這是一個簡單的勾點、可成功將訊息寫入標準輸出和標準錯誤。

```
#!/bin/sh

# success_sample.sh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#
```

```
# log something to stdout
info "running success_sample.sh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

簡單的成功範例（Bash版本）

這是一個簡單的攔截、成功將訊息寫入標準輸出和標準錯誤（寫入Bash）。

```
#!/bin/bash

# success_sample.bash
#
# A simple noop success hook script for testing purposes.
#
# args: None


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
```

```
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.bash"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

簡單的成功範例（zsh版本）

這是一個簡單的勾點範例、可成功將訊息寫入標準輸出和標準錯誤、並寫入Z Shell。

```
#!/bin/zsh

# success_sample.zsh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
```

```
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}

#
# main
#

# log something to stdout
info "running success_sample.zsh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

成功的引數範例

下列範例示範如何在攔截中使用args。

```
#!/bin/sh

# success_sample_args.sh
#
# A simple success hook script with args for testing purposes.
#
# args: Up to two optional args that are echoed to stdout

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}
```

```
#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample_args.sh"


# collect args
arg1=$1
arg2=$2

# output args and arg count to stdout
info "number of args: $#"
info "arg1 ${arg1}"
info "arg2 ${arg2}"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

快照前/快照後掛機範例

以下範例說明如何將相同的指令碼同時用於快照前和快照後掛勾。

```
#!/bin/sh

# success_sample_pre_post.sh
#
# A simple success hook script example with an arg for testing purposes
# to demonstrate how the same script can be used for both a prehook and
posthook
#
# args: [pre|post]

# unique error codes for every error case
ebase=100
eusage=$((ebase+1))
ebadstage=$((ebase+2))
epre=$((ebase+3))
epost=$((ebase+4))


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}
```

```
#
# Would run prehook steps here
#
prehook() {
    info "Running noop prehook"
    return 0
}

#
# Would run posthook steps here
#
posthook() {
    info "Running noop posthook"
    return 0
}


#
# main
#

# check arg
stage=$1
if [ -z "${stage}" ]; then
    echo "Usage: $0 <pre|post>"
    exit ${eusage}
fi

if [ "${stage}" != "pre" ] && [ "${stage}" != "post" ]; then
    echo "Invalid arg: ${stage}"
    exit ${ebadstage}
fi

# log something to stdout
info "running success_sample_pre_post.sh"

if [ "${stage}" = "pre" ]; then
    prehook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during prehook"
    fi
fi

if [ "${stage}" = "post" ]; then
    posthook
```

```
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during posthook"
    fi
fi

exit ${rc}
```

故障範例

下列範例示範如何處理攔截式故障。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
```

```
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

詳細故障範例

下列範例示範如何以更詳細的記錄功能來處理掛機中的失敗。

```
#!/bin/sh

# failure_sample_verbose.sh
#
# A simple failure hook script with args for testing purposes.
#
# args: [The number of lines to output to stdout]


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
```

```
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_verbose.sh"


# output arg value to stdout
linecount=$1
info "line count ${linecount}"

# write out a line to stdout based on line count arg
i=1
while [ "$i" -le ${linecount} ]; do
    info "This is line ${i} from failure_sample_verbose.sh"
    i=$(( i + 1 ))
done

error "exiting with error code 8"
exit 8
```

退出程式碼範例失敗

下列範例示範攔截失敗、並顯示結束程式碼。

```
#!/bin/sh
```

```
# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
```

```
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

失敗後成功範例

下列範例示範第一次執行時發生掛機故障、但在第二次執行後成功。

```
#!/bin/sh

# failure_then_success_sample.sh
#
# A hook script that fails on initial run but succeeds on second run for
testing purposes.
#
# Helpful for testing retry logic for post hooks.
#
# args: None
#

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
```

```
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_success sample.sh"


if [ -e /tmp/hook-test.junk ] ; then
    info "File does exist.  Removing /tmp/hook-test.junk"
    rm /tmp/hook-test.junk
    info "Second run so returning exit code 0"
    exit 0
else
    info "File does not exist.  Creating /tmp/hook-test.junk"
    echo "test" > /tmp/hook-test.junk
    error "Failed first run, returning exit code 5"
    exit 5
fi
```

版權資訊

Copyright © 2023 NetApp, Inc. 版權所有。台灣印製。非經版權所有人事先書面同意，不得將本受版權保護文件的任何部分以任何形式或任何方法（圖形、電子或機械）重製，包括影印、錄影、錄音或儲存至電子檢索系統中。

由 NetApp 版權資料衍伸之軟體必須遵守下列授權和免責聲明：

此軟體以 NETAPP「原樣」提供，不含任何明示或暗示的擔保，包括但不限於有關適售性或特定目的適用性之擔保，特此聲明。於任何情況下，就任何已造成或基於任何理論上責任之直接性、間接性、附隨性、特殊性、懲罰性或衍生性損害（包括但不限於替代商品或服務之採購；使用、資料或利潤上的損失；或企業營運中斷），無論是在使用此軟體時以任何方式所產生的契約、嚴格責任或侵權行為（包括疏忽或其他）等方面，NetApp 概不負責，即使已被告知有前述損害存在之可能性亦然。

NetApp 保留隨時變更本文所述之任何產品的權利，恕不另行通知。NetApp 不承擔因使用本文所述之產品而產生的責任或義務，除非明確經過 NetApp 書面同意。使用或購買此產品並不會在依據任何專利權、商標權或任何其他 NetApp 智慧財產權的情況下轉讓授權。

本手冊所述之產品受到一項（含）以上的美國專利、國外專利或申請中專利所保障。

有限權利說明：政府機關的使用、複製或公開揭露須受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中的「技術資料權利 - 非商業項目」條款 (b)(3) 小段所述之限制。

此處所含屬於商業產品和 / 或商業服務（如 FAR 2.101 所定義）的資料均為 NetApp, Inc. 所有。根據本協議提供的所有 NetApp 技術資料和電腦軟體皆屬於商業性質，並且完全由私人出資開發。 美國政府對於該資料具有非專屬、非轉讓、非轉授權、全球性、有限且不可撤銷的使用權限，僅限於美國政府為傳輸此資料所訂合約所允許之範圍，並基於履行該合約之目的方可使用。除非本文另有規定，否則未經 NetApp Inc. 事前書面許可，不得逕行使用、揭露、重製、修改、履行或展示該資料。美國政府授予國防部之許可權利，僅適用於 DFARS 條款 252.227-7015(b)（2014 年 2 月）所述權利。

商標資訊

NETAPP、NETAPP 標誌及 http://www.netapp.com/TM 所列之標章均為 NetApp, Inc. 的商標。文中所涉及的所有其他公司或產品名稱，均為其各自所有者的商標，不得侵犯。